

Triple Peaks

La Cordillera Oriental es un rango de montañas de los Andes que atraviesa Bolivia. Consiste en una sucesión de N cimas de montañas, numeradas de 0 a $N - 1$. La **altura** de la cima i ($0 \leq i < N$) es $H[i]$, la cual es un entero entre 1 y $N - 1$, inclusive.

Para cualesquiera dos cimas i y j donde $0 \leq i < j < N$, la **distancia** entre ellos se define como $d(i, j) = j - i$.

De acuerdo a antiguas leyendas incas, un trío de cimas es **mítico** si cumple la siguiente propiedad especial: las alturas de las tres cimas **coinciden** con sus distancias dos a dos **sin importar el orden**.

Formalmente, un trío de cimas (i, j, k) es mítico si

- $0 \leq i < j < k < N$, y
- las alturas $(H[i], H[j], H[k])$ coinciden con las distancias dos a dos $(d(i, j), d(i, k), d(j, k))$ sin importar el orden.

Por ejemplo, para los índices 0, 1, 2 las distancias dos a dos son (1, 2, 1), así que las alturas $(H[0], H[1], H[2]) = (1, 1, 2)$, $(H[0], H[1], H[2]) = (1, 2, 1)$, y $(H[0], H[1], H[2]) = (2, 1, 1)$ todas coinciden, pero las alturas $(H[0], H[1], H[2]) = (1, 2, 2)$ no.

Este ejercicio consiste en dos partes, con cada subtarea asociada o con **Parte I** o con **Parte II**. Puedes resolver las subtareas en cualquier orden. Es decir, **no** es requerido completar toda la Parte I antes de intentar la Parte II.

Parte I

Dada la descripción del rango de montañas, tu tarea consiste en contar el número de tríos míticos.

Detalles de implementación

Debes implementar la siguiente función.

```
long long count_triples(std::vector H)
```

- H : vector de longitud N , representando las alturas de las cimas.
- La función se llama exactamente una vez para cada caso de prueba.

La función debe devolver un entero T , el número de tríos míticos en el rango de montañas.

Restricciones

- $3 \leq N \leq 200\,000$
- $1 \leq H[i] \leq N - 1$ para cada i tal que $0 \leq i < N$.

Subtareas

La Parte I otorga un total de 70 puntos.

Subtarea	Puntuación	Restricciones adicionales
1	8	$N \leq 100$
2	6	$H[i] \leq 10$ para cada i tal que $0 \leq i < N$.
3	10	$N \leq 2000$
4	11	Las alturas son no-decrecientes. Esto es, $H[i - 1] \leq H[i]$ para cada i tal que $1 \leq i < N$.
5	16	$N \leq 50\,000$
6	19	Sin restricciones adicionales.

Ejemplo

Considera la siguiente llamada.

```
count_triples([4, 1, 4, 3, 2, 6, 1])
```

Hay 3 tríos míticos en el rango de montañas:

- Para $(i, j, k) = (1, 3, 4)$, las alturas $(1, 3, 2)$ coinciden con las distancias dos a dos $(2, 3, 1)$.
- Para $(i, j, k) = (2, 3, 6)$, las alturas $(4, 3, 1)$ coinciden con las distancias dos a dos $(1, 4, 3)$.
- Para $(i, j, k) = (3, 4, 6)$, las alturas $(3, 2, 1)$ coinciden con las distancias dos a dos $(1, 3, 2)$.

Por lo tanto, la función debe devolver 3.

Fíjate que los índices $(0, 2, 4)$ no forman un trío mítico, dado que las alturas $(4, 4, 2)$ no coinciden con las distancias dos a dos $(2, 4, 2)$.

Parte II

Tu misión consiste en construir rangos de montañas con muchos tríos míticos. Esta parte consiste en 6 subtareas **sólo salida** ("output only") con **puntuación parcial**.

En cada subtarea, se te dan dos enteros positivos M y K , y debes construir un rango de montañas con M cimas **como máximo**. Si tu solución contiene **al menos** K tríos míticos, recibirás toda la puntuación para esta subtarea. En cualquier otro caso, tu puntuación será proporcional al número de tríos míticos que contenga tu solución.

Fíjate que tu solución debe consistir de un rango válido de montañas. Específicamente, supón que tu solución tiene N cimas (N debe satisfacer $3 \leq N \leq M$). Entonces, la altura de la cima i ($0 \leq i < N$), denotada $H[i]$, debe ser un entero entre 1 y $N - 1$, inclusive.

Detalles de implementación

Hay dos métodos para enviar tu solución, y puedes usar cualquiera de los dos para cada subtarea:

- **Archivo de salida**
- **Llamada a función**

Para enviar tu solución mediante un **archivo de salida**, crea y envía un archivo de texto con el siguiente formato:

```
N
H[0] H[1] ... H[N-1]
```

Para enviar tu solución mediante una **llamada a función**, debes implementar la siguiente función.

```
std::vector construct_range(int M, int K)
```

- M : el número máximo de cimas.
- K : el número deseado de tríos míticos.
- Esta función es llamada exactamente una vez para cada subtarea.

La función debe devolver un vector H de longitud N , representando las alturas de las cimas.

Subtareas y puntuación

La Parte II otorga un total de 30 puntos. Para cada subtarea, los valores de M y K son fijos y dados en la siguiente tabla:

Subtarea	Puntuación	M	K
7	5	20	30
8	5	500	2000
9	5	5000	50 000
10	5	30 000	700 000
11	5	100 000	2 000 000
12	5	200 000	12 000 000

Para cada subtarea, si tu solución no es un rango de montañas válido, tu puntuación será 0 (informado como `Output isn't correct` en CMS).

En cualquier otro caso, sea T el número de trios míticos en tu solución. Entonces, tu puntuación para la tarea es:

$$5 \cdot \min \left(1, \frac{T}{K} \right).$$

Sample Grader

Las Partes I y II usan el mismo programa como sample grader, con la distinción entre las dos partes determinada por la primera línea de la entrada.

Formato de entrada para la Parte I:

```
1
N
H[0] H[1] ... H[N-1]
```

Formato de salida para la Parte I:

```
T
```

Formato de entrada para la Parte II:

```
2
M K
```

Formato de salida para la Parte II:

N

H[0] H[1] ... H[N-1]

Fíjate que la salida del sample grader cumple con el formato requerido para el archivo de salida en la Parte II.